IN3043 Functional Programming
Coursework

There is a single coursework in this module, counting for 30% of the overall module mark. This coursework is due at **5pm on Sunday 3rd December**. As with all modules, this deadline is hard. (See "Submission" below for the procedure for applying for extensions.)

This coursework is to be done as a **pair programming** task. More details are given in a later section.

The aim of this coursework is to allow you to demonstrate effective use of the Haskell language, containers and higher-order functions to construct solutions that are correct, clear and concise. This task does not require any material presented after session 6, and I do not expect to see it in your solutions.

Background

Suppose you are told that numbers are being used as code for fruit. Each fruit is denoted by a single number, which is used exclusively for that fruit. After some investigation, you discover the following possibilities:

- 1 might be apple
- 2 might be banana
- 3 might be orange
- 1 might be banana
- 3 might be apple
- 3 might be banana

We assume that there are no other possibilities. In this case, this is sufficient information to work out the coding:

- Since there is only one possibility for 2, it must be banana. This means none of the other numbers can be banana, so we have:
  - 1 might be apple
  - 3 might be apple
  - 3 might be orange
- Similarly, we now know that 1 must be apple. Removing that possibility, we have
  - 3 might be orange
- Thus 3 must be orange.

A general property of such coding problems is that is they have a unique solution, there will always be at least one number that has only one possibility, so we can deduce a corresponding name as above.

The task

You are to write a Haskell function

```
solve :: [(Int, String)] -> [(Int, String)]
```

using the strategy outlined above to solve coding problems of this type, and to return the solution in ascending order of the first component of the pair.

For example, the above example could be expressed by a value

```
example :: [(Int, String)]
example = [
    (1, "apple"), (1, "banana"), (2, "banana"),
    (3, "apple"), (3, "banana"), (3, "orange")]
```

Then evaluating **`solve example`** would yield

```
[(1, "apple"), (2, "banana"), (3, "orange")]
```

Your solution should work for any problem of this type that has a unique solution. You may assume that your function will never be applied to a problem that does not have a unique solution. You should not assume that the input will be any particular order, i.e. the above example should yield the same answer for any rearrangement of the argument list.

There are many different ways of implementing this functions, so I am not expecting to see identical or strongly similar answers.

Pair working

You are expected to work on this coursework in pairs. School policy is that students cannot choose their own pairs. I have allocated pairs, grouping students from the same lab group with similar levels of Moodle activity on this module. Please contact your assigned partner and arrange to meet to work on this task together, for example in your timetabled lab session or in the Ada Lovelace area. Contact me if there are problems (but remember the School policy mentioned above).

After submission of the work, each student should go to the *Rate contributions to the coursework* entry to give an indication of their view of the contributions to the work. Ideally, both members of a pair will agree that they made roughly equal contributions, in which case they will each receive the same mark. If not, I will interview each member individually regarding the submission, to determine appropriate adjustments of the mark.

You are strongly encouraged to generate an agreed record of your pair meetings, with dates, what was decided and actions for work agreed upon. These can then be used to inform your *Rate contributions to the coursework* entry and if necessary be called upon should you have unequal or disagreed contributions.

Submission

You should submit a single Haskell source file, defining the function **`solve`** as above, via Moodle by the due date. Only one member of each pair need submit the file. After submission, each member of the pair should also complete the *Rate contributions to the coursework* entry.

If you believe that you have extenuating circumstances that justify an extension, you should

- submit your work to `smcse-extension@city.ac.uk` within one week of the deadline (i.e. by 5pm Sunday 10th December), including your student number, course, module code and the original due date, **and**
- apply for an extension through the Extenuating Circumstances process.

As not all claims for extenuating circumstances meet the criteria, you can also submit on Moodle what you have by the deadline, as a fallback in case your claim is rejected.

Marking criteria

A correct solution will achieve at least 60%, regardless of style. To earn a mark in the first class range (70% or more), a solution should make effective use of the language and containers to achieve clarity and avoid unnecessary repetition and complexity.

Advice

The neatest solutions to this task will probably use containers, as introduced in session 6. You should probably do some of the exercises for that week, at least the first two, before attempting this task. Try to use operations on whole containers.

As noted above, this task should not require any material covered after session 6. In particular, it does not require recursion. You can generate the sequence of reduced problems using functions covered in session 5.

Caution

We expect that your submissions should be your own work. You must not discuss the details of this coursework with students outside your pair, nor can you ever share your code (in any form) with them. You must not use generative AI tools, or code found online. Any of these this things may be treated as academic misconduct.

Help and feedback

Please ask general questions about the coursework on the Moodle discussion board. Queries by email will be answered on that board, so that everyone gets the same information.

We can view and discuss draft solutions in the weekly lab session. You can also use my drop-in hours or make an appointment by email to see me at other times.

Sample solutions will be published on Moodle after the deadline. Marks and comments on your code will be returned via Moodle.